CLAWW

How the app works, end to end

A complete technical walkthrough of CLAWW: how workouts are generated, how nutrition targets and meal logging work, and how the adaptive system changes your plan based on what you actually do.



Covers
App version 1.0.0 · Expo SDK 54 · Supabase backend

Source of truth
Every number in this document is taken from the code, not from a spec

Prepared
29 September 2026

Audience
Founders, engineers, and anyone who needs to explain the product

Contents

1 The big picture
What CLAWW is, the architecture, and the four rules the engine follows

2 The user journey
Sign-up, onboarding, workout setup, nutrition setup, and the daily loop

3 Where the data lives
Every table, what writes to it, and what reads from it

4 How a workout plan is generated
The 12-step pipeline from your profile to a saved plan

5 Doing a workout
The live session, set logging, rest, day rotation, manual additions, calorie burn

6 Nutrition
Calorie and macro targets, meal logging by text or photo, suggestions, water

7 The adaptive system
Recovery, performance trends, feedback, and how they change the next plan

8 Progress, XP and streaks
How points, levels, streaks, achievements and the Tracker work

9 Home, sensors and reminders
The dashboard rings, step counting, and notifications

10 Reliability, safety and cost control
Model fallback, failure logging, usage caps, privacy

11 Honest limits and what's next
What the system does not do yet

A Appendix
File map and glossary

1 The big picture

CLAWW is a fitness app that builds a personal workout plan and nutrition targets, logs what you actually do, and uses those logs to adjust the next plan.

What happens, in one paragraph

You create an account and answer a short set of questions about your body, goal, experience, equipment and schedule. The server turns those answers into hard numbers — how many days to train, how many sets and reps per exercise — using fixed exercise-science tables. A language model then picks *which* exercises to put in the plan, but only from a pre-filtered list and only inside those numbers. Every answer the model gives is checked, cleaned and clamped before it is saved. As you train, eat, sleep and drink water, the app logs it. The next time a plan is generated, those logs shift the numbers: poor recovery trims volume, consistently beating your reps raises the rep range, telling the app a plan was "too hard" lowers it.

Architecture

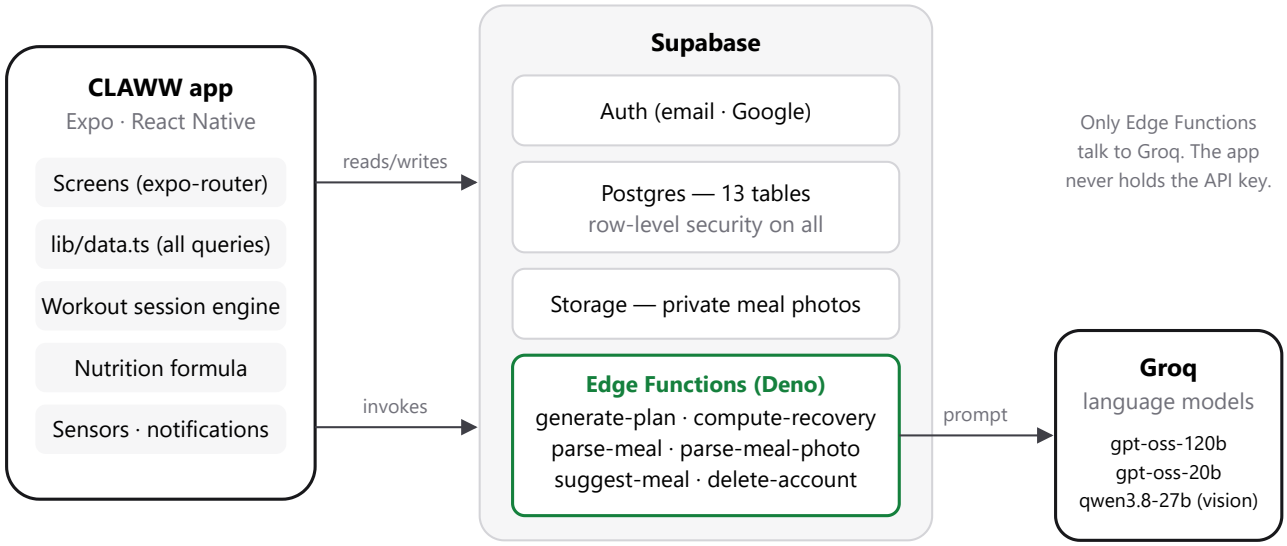


Figure 1 — The app talks to Supabase directly for ordinary reads and writes (protected by row-level security), and calls Edge Functions for anything that needs a language model or elevated permissions.

Layer	Technology	What it is responsible for
App	Expo SDK 54, React Native, expo-router, TypeScript	Every screen, the live workout player, the nutrition formula, step counting, local reminders, and the ring graphics (Skia)
Auth	Supabase Auth	Email/password and Google sign-in, password reset by deep link
Database	Postgres with row-level security	All user data. Every table is locked so a user can only read and write their own rows
Server logic	Supabase Edge Functions (Deno)	Plan generation, recovery score, meal estimation from text and photo, meal suggestions, account deletion, health check
Language models	Groq API	Choosing exercises, naming meals, and estimating nutrition — never computing the numbers the app enforces

The four rules the engine follows

These come from the project's own rules file and are enforced in the code. Everything else in this document follows from them.

Rule	What it means in practice
1. The model never computes numbers.	Sets, reps, training days, calorie targets and recovery scores are all computed by fixed formulas. The model only chooses names (exercises, meals) inside limits already calculated for it. Its output is validated with a schema, then clamped, and a hard-coded fallback is used if it fails.
2. Generation is invisible.	The interface never labels anything "AI-generated". Generation is plumbing, the same way a camera app doesn't label its noise reduction.
3. Row-level security on everything.	Every table has a policy restricting rows to their owner. The app talks to the database with the user's own login token, so even a bug in the app cannot read another user's data.
4. One schema file.	The whole database is defined in `database/schema.sql`, written so it can be re-run safely at any time.

2 The user journey

From the first launch to a normal day in the app, in the order a new user experiences it.

Stage 1 — Account and basic profile

1. **Welcome and sign-up**

   Email and password, or Google. A database trigger (`handle_new_user`) creates the user's `profiles` row automatically the moment the account exists.

   screens/auth/welcome.tsx, sign-up.tsx, sign-in.tsx

2. **Name → age → gender**

   Three short screens. Age and gender feed the calorie formula and the age-based safety limits on training volume.

   screens/onboarding/name.tsx, age.tsx, gender.tsx

3. **Reveal and medical disclaimer**

   "You're in, [name]." The **Enter CLAWW** button stays disabled until the user ticks "I understand this isn't medical advice". The time of that acknowledgement is saved to `profiles.medical_disclaimer_acknowledged_at` .

   screens/onboarding/reveal.tsx

Stage 2 — Workout setup (8 steps)

Launched from Home ("Generate your first plan"). Single-choice steps move forward the instant you tap an option; multi-choice steps have a Continue button.

#	Screen	Asks for	Options	Used by
1	Basics	Height, weight	cm, kg	Nutrition formula, calorie burn
2	Equipment	What you can train with	none (bodyweight) · home · gym	Exercise filter
3	Modality	Kinds of training (multi-select)	strength · cardio · mobility · yoga	Exercise filter
4	Goal	What you want	muscle gain · fat loss · endurance · maintenance · flexibility	Rep ranges, calories, protein
5	Experience	Training history	beginner · intermediate · advanced	Set cap, training days
6	Activity	Daily activity outside training	sedentary · moderate · active	Training days, calorie multiplier
7	Schedule optional	Which weekdays, and a split	Mon–Sun chips; auto · full body · upper/lower · push/pull/legs	Day count, day names, split
8	Review	Injuries or limitations (free text), then Generate	e.g. "bad left knee"	Injury exclusion filter

On **Generate**, the answers are saved (`saveWorkoutIntake`): height, weight, goal, experience and equipment go into their own columns on `profiles` ; modalities, activity level, injuries, training days and split go into `profiles.personalization_profile.workoutDefaults` . Then the `generate-plan` function runs (Section 4). The user earns **50 XP**.

Stage 3 — Nutrition setup (5 steps)

Launched from the Nutrition tab. Unlike workout setup, it pre-fills everything the app already knows (height, weight, age, goal, activity) so the user usually just confirms.

#	Screen	Asks for
1	Details	Height, weight, age, gender
2	Goal	Same five goals as workout setup
3	Activity	sedentary · moderate · active
4	Dietary	Restrictions (multi-select), e.g. vegetarian, dairy-free
5	Review	Shows the calculated calories/protein/carbs/fat, then saves them

The calculation is instant and happens on the phone (Section 6). Targets are saved to `personalization_profile.nutritionDefaults` ; restrictions to `personalization_profile.dietaryRestrictions` .

Stage 4 — The daily loop



Figure 2 — A normal day. None of these steps is mandatory; each one simply adds signal.

The five tabs

Tab	What it shows
Home	Recovery score and band, the three rings (recovery, sleep, nutrition), daily goal %, steps, calories burned, streak, XP, sleep check-in, water widget
Workouts	The current plan by day, today's suggested day, today's stats (progress, volume, calories burned), completed/skipped calendar, regenerate with a reason, add an exercise, log an extra session
Nutrition	Calorie ring, macros eaten vs target, meal timeline (Breakfast, Lunch, Snack, Dinner), log/suggest a meal, water
Tracker	CLAWW Score, streak, workouts, volume, goals and nutrition summary. Fully unlocks after 3 workout days
More	Profile, settings (usage limits, reminders), privacy and data, about, help

3 Where the data lives

Thirteen tables, one view, one storage bucket. Every row belongs to one user; deleting the account deletes all of it automatically.

Table	Holds	Written by	Read by
`profiles`	Height, weight, age, gender, goal, experience, equipment, and `personalization_profile` (JSON: workout defaults, nutrition targets, dietary restrictions)	Onboarding, setup flows, settings	Nearly everything
`workouts`	Each generated plan as JSON: days → exercises → sets/reps	generate-plan; manual "add exercise"	Workouts tab, session
`exercises`	The catalog: 183 exercises with modality, muscle group, equipment and MET value	Seed file (admin)	generate-plan, exercise picker, calorie burn
`workout_logs`	One row per completed set: exercise, set number, reps and weight prescribed vs achieved, time, optional duration	Live session, extra-session logging	Adaptive system, volume, streak, Tracker
`workout_day_events`	"Day X completed/skipped on date Y"	Session finish	Next-day suggestion, calendar
`sleep_logs`	Hours, bedtime, wake time	Sleep check-in	Recovery score, streak, achievements
`meal_logs`	Description, calories, protein, carbs, fat, meal type, optional photo path	Meal logging (after review)	Nutrition tab, recovery, streak
`water_logs`	Millilitres per entry	Water widget	Recovery score, Home
`xp_events`	Append-only ledger: amount + reason	Every XP-earning action	`user_xp` view (sum)
`generation_usage`	Per-user, per-day counters of model calls	A database function (atomic)	Usage caps, Settings
`generation_failures`	Every failed model call with a reason	Edge Functions	Operations/debugging
`user_stats`, `nutrition_logs`	Older tables kept for compatibility	—	Data export
Storage: `meal-photos`	Private bucket, files stored under the user's own ID folder	Meal logging	"Eaten this before?" strip (1-hour signed links)

A DETAIL THAT MATTERS LATER

The `sets` column on `workout_logs` holds the *set number* (1, 2, 3…), not a count of sets. Each row already is one set. An earlier version of the volume calculation multiplied by this number and overstated volume; that has been fixed (Section 5).

4 How a workout plan is generated

The `generate-plan` Edge Function. Twelve steps turn a profile and a history into a saved plan. Steps 1–7 and 10–12 are fixed code; only step 8 involves a language model.

1. **Authenticate the caller**
   The function reads the user's login token from the request and confirms who they are. The user ID is never taken from the request body.

2. **Gather everything in parallel**
   Seven queries at once: the profile; the latest sleep log; the latest workout set; the last 10 workout sets; the latest saved plan; today's water; and whether any meal was logged today.

3. **Compute the recovery score**
   Sleep, time since last session, last session's load, hydration and nutrition logging combine into a 0–100 score and a band: Low, Moderate or High. (Full formula in Section 7.)

4. **Double-tap protection**
   If a plan was generated less than **30 seconds** ago and no reason was given, the function returns that plan instead of making another model call. A regeneration with a reason always goes through.

5. **Filter the exercise catalog**
   Keep only exercises in the user's chosen modalities and allowed equipment tier, then remove anything that conflicts with reported injuries.

6. **Compute the base rep range**
   From goal, capped by experience, tightened by age.

7. **Apply the adaptive adjustments**
   Recovery band moves the maximum sets by one; performance trend or explicit feedback moves the reps. (Section 7.)

8. **Ask the model to choose exercises**
   The model receives a prompt containing the exact day count, set/rep limits, injury note, split and day names, plus the filtered catalog. It must reply with JSON only.

9 **Validate the shape**

The reply is checked against a strict schema. If it doesn't match, the failure is logged and a built-in starter plan is used instead.

10 **Ground every exercise in the catalog**

Any exercise name the model invented (not in the list it was given) is dropped. Days left empty are dropped. Each surviving exercise gets the real catalog ID.

11 **Clamp to the computed limits**

Every exercise's sets and reps are forced into range and the number of days is cut to the computed count — regardless of what the model returned.

12 **Save and return**

The plan is inserted into `workouts`. The response includes the recovery result and whether the plan was genuinely generated or a fallback.

Step 5 in detail — which exercises are even allowed

EQUIPMENT TIERS

Higher access includes the lower tiers: a gym-goer can still do bodyweight moves, but a bodyweight-only user never sees gym machines.

User's equipment	Allowed catalog equipment	Exercises available
none (bodyweight)	none	119
home	none + home	155
gym	none + home + gym	183
(missing/unknown)	treated as gym	183

THE CATALOG

Modality	Bodyweight	Home	Gym	Total
Strength	30	13	14	57
Cardio	21	11	10	42
Mobility	32	8	2	42
Yoga	36	4	2	42
Total	**119**	**36**	**28**	**183**

Each exercise also stores a **MET value** (metabolic equivalent — how hard it is relative to sitting still), which is later used for calorie burn.

INJURY EXCLUSIONS

If the free-text injury field mentions a keyword, every exercise whose name matches the pattern is removed *before* the model sees the list, so it cannot choose them.

If injuries mention…	Exercises removed if the name contains…
knee	squat, lunge, jump
back, spine, spinal	deadlift, squat, row, good morning
shoulder	overhead press, bench press, push-up
wrist	push-up, plank
ankle, foot	jump, lunge, run, jog
hip	squat, lunge, deadlift

LIMIT

This is a keyword filter, not a clinical assessment. "My knees are fine" would still trigger the knee rule. Replacing it with structured movement-pattern contraindications is on the roadmap (Ship Phase 10.1).

Step 6 in detail — the base rep range

STARTING POINT: THE GOAL

Goal	Sets	Reps	Reasoning
Muscle gain	3–5	6–12	Classic hypertrophy range
Fat loss	2–4	12–20	Higher reps, more work per set
Endurance	2–3	15–25	Muscular endurance
Maintenance (also the default)	2–4	8–15	Middle ground
Flexibility	1–3	20–60	Holds and flows (reps often mean seconds)

THEN: THE EXPERIENCE CAP ON SETS

Experience	Max sets per exercise
Beginner	3
Intermediate (default)	4
Advanced	5

THEN: AGE SAFETY LIMITS

Age	Max sets capped at	Minimum reps raised to	Why
65 and over	3	10	Avoid heavy, near-maximal low-rep work
50–64	4	8	Same, less strictly

```
maxSets = min(goal.maxSets, experienceCap)      then age cap if applicable
minReps = goal.minReps                             then raised by age rule if applicable
minSets = min(goal.minSets, maxSets)
maxReps = max(goal.maxReps, minReps)
```

HOW MANY TRAINING DAYS

If the user picked specific weekdays, the day count is simply how many they picked. Otherwise it comes from this table:

Experience \ Activity	Sedentary	Moderate	Active
Beginner	2	3	3
Intermediate	3	4	4
Advanced	3	4	5

Users aged 55+ are capped at 4 days.

Step 8 in detail — what the model is told

The model is framed as a certified personal trainer and given a list of hard constraints that are already computed:

- Generate **exactly N** training days.
- Name days with the real weekdays chosen (e.g. "Monday, Wednesday, Friday"), or "Day 1, Day 2…" if none were chosen.
- Every exercise must use **X–Y sets** and **A–B reps**.
- The injury note, and an instruction not to work around the filter.
- If a split was chosen: Full Body (every day trains the whole body), Upper/Lower (strictly alternating), or Push/Pull/Legs (strictly cycling).
- If the user regenerated with "wrong focus": choose a meaningfully different split and selection.

Within those constraints it uses judgment: a sensible split for the goal and equipment, landing lower in each range when recovery is Low and higher when High. Alongside the prompt it receives the recovery result, the personalization profile, the last 10 sets, and the filtered catalog.

The expected reply shape is:

```
{ "days": [ { "day": "Monday", "focus": "Push",
             "exercises": [ { "exerciseId": "…", "name": "Push-Up", "sets": 3, "reps": 10 }
] } ],
  "notes": "…" }
```

Steps 9–11 in detail — the safety net

Check	What it catches	What happens
Schema validation (Zod)	Missing fields, wrong types, zero days, zero exercises, non-positive numbers	Failure logged as `validation_failed`; starter plan used
Catalog grounding	Invented exercises, misspelt names	Dropped (matching ignores case and extra spaces). Empty days dropped. If nothing survives: logged as `grounding_emptied`, starter plan used
Clamping	Sets/reps outside the computed range; too many days	Forced into range; extra days cut
Model unavailable	Every model in the chain failed	Failure logged with its type; starter plan used

THE STARTER PLAN (FALLBACK)

Three days, all bodyweight, so it works for anyone: **Full Body** (bodyweight squat 3×15, push-up 3×10, plank 3×30); **Active Recovery** (walking, stretching); **Full Body** (lunges 3×12, glute bridge 3×15, bird dog 3×10). Even the fallback is clamped to the user's computed ranges before saving.

Worked example — Riya, 25, beginner, muscle gain, bodyweight only, moderately active, no weekdays chosen

Stage	Calculation	Result
Goal range	muscle gain	3–5 sets, 6–12 reps
Experience cap	beginner → max 3 sets	3–3 sets, 6–12 reps
Age	25 — no adjustment	3–3 sets, 6–12 reps
Recovery	score 77 → High → +1 set, but capped at beginner max of 3	3–3 sets, 6–12 reps
Trend	last 10 sets averaged 110% of prescribed reps (≥105%) → up by 2	3 sets, **8–14 reps**
Days	beginner × moderate	**3 days**
Catalog	all 4 modalities, equipment "none", no injuries	119 exercises offered

The model then picks exercises from those 119 to fill 3 days, and whatever it returns is clamped to exactly 3 sets of 8–14 reps.

Cost and abuse protection on generation

- **30-second cooldown** for repeated taps without a reason (step 4).
- **20 plan generations per user per day.** The count is checked and incremented in one atomic database operation right before the model call, so a cached answer never uses up a slot and two rapid requests cannot both slip through. Hitting the cap returns a clear "try again tomorrow" message rather than a silent fallback.

5 Doing a workout

What happens between tapping Start and the completion screen, and how the Workouts tab decides what to show.

Which day is "today"

A plan's days repeat indefinitely. The app suggests the day **after** whichever day was most recently completed or skipped, wrapping back to the first day at the end. With no history for the current plan, it suggests the first day.

```
suggested = (index of most recently completed-or-skipped day + 1) mod number_of_days
```

The live session

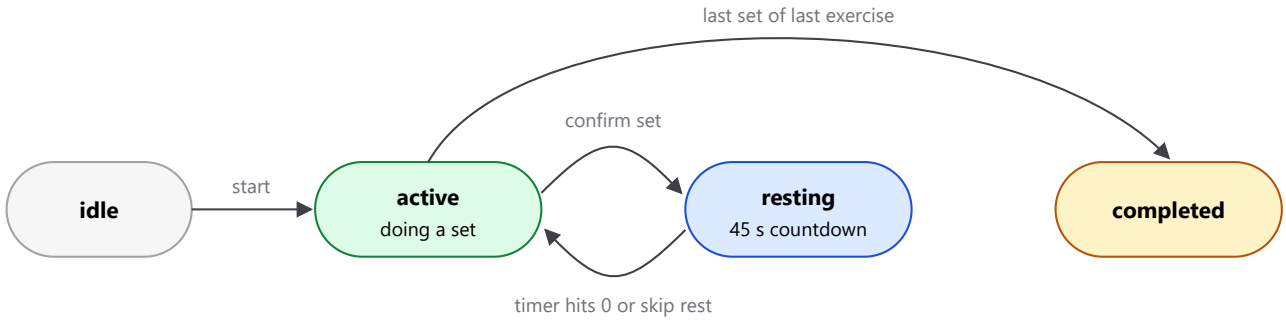


Figure 3 — The session state machine (lib/workoutSession.tsx). Pause freezes both the session clock and the rest timer.

1 **Start**

Counters reset, the session clock starts, the player opens on exercise 1, set 1.

2 **Confirm the set**

After each set, a confirm card appears pre-filled with the **prescribed** reps and the current weight. The user corrects them to what they **actually** did. This is the single most important input to the adaptive system — it is how the app learns the difference between the plan and reality.

3 **Log it**

One row goes into `workout_logs` with exercise name and catalog ID, set number, reps prescribed vs achieved, and weight prescribed vs achieved. **+5 XP**, shown instantly.

4 **Rest**

A 45-second rest starts. The rest screen shows the set just finished ("4/4 ✓"), not the upcoming one, so the final set of an exercise is visibly confirmed.

5 **Carry the weight forward**

The next set of the same exercise starts from the weight actually lifted. A new exercise starts from 0.

6 **Finish**

After the last set of the last exercise (or skipping the last exercise), the day is recorded as **completed** in `workout_day_events`, and the summary screen shows time, sets, reps, exercises and XP earned.

The completion screen also handles two one-time prompts. After the **first** completed workout it offers daily reminders. Otherwise, after the **third**, it may request an app-store rating — never both on the same screen, and each only ever once.

Changing the plan by hand

Feature	How it works
+ Add Exercise	Search the same catalog, filtered to the user's equipment tier. The chosen exercise is appended to a specific day in the saved plan.
Log a Session	Record training that wasn't in the plan, like an extra run. Strength: reps and weight. Cardio: duration in minutes. It is written to `workout_logs` just like a planned set, so it counts towards streak, volume and workout days. +5 XP.
Regenerate	Pick a reason — Too hard, Too easy, or Wrong focus — and a new plan is generated. The reason is fed straight into the adaptive system (Section 7).

Today's stats on the Workouts tab

VOLUME

```
volume = Σ over every logged set:  reps × weight
         (achieved values where logged, prescribed values otherwise)
```

CALORIES BURNED DURING TRAINING

Uses the standard MET method, with the duration of each set measured from real timestamps, not guessed.

```
calories = Σ over today's sets:  MET × body_weight_kg × (seconds ÷ 3600)

  MET      = the exercise's catalog MET value, or 5.0 if missing (ACSM "weight lifting,
general")
  seconds  = time since the previous set's completion, capped at 600 (10 min)
             the first set of the day is counted as 60 s
  weight   = profile weight, or 70 kg if not set
```

Worked example — 70 kg user, three sets of push-ups (MET 3.8)

Set	Time since previous	Calculation	kcal
1	first set → 60 s	3.8 × 70 × 60/3600	4.4
2	95 s	3.8 × 70 × 95/3600	7.0
3	110 s	3.8 × 70 × 110/3600	8.1
Total (rounded)			**20**

6 Nutrition

Targets are pure maths done on the phone. Meal estimates come from a language model, but always pass through a review screen before anything is saved.

6.1 Daily targets

Calculated instantly in `lib/nutrition.ts` using the Mifflin-St Jeor equation, the most widely used formula for resting energy expenditure.

STEP 1 — BASAL METABOLIC RATE (BMR): ENERGY USED AT COMPLETE REST

```
Male:    BMR = 10 × weight_kg + 6.25 × height_cm − 5 × age + 5
Female:  BMR = 10 × weight_kg + 6.25 × height_cm − 5 × age − 161
Other / not given:  the average of the two
```

STEP 2 — TOTAL DAILY ENERGY EXPENDITURE (TDEE)

Activity	Multiplier
Sedentary	× 1.20
Moderate	× 1.45
Active	× 1.70

```
TDEE = BMR × multiplier
```

STEP 3 — ADJUST FOR THE GOAL

Goal	Adjustment
Fat loss	−500 kcal
Muscle gain	+300 kcal
Everything else	0

```
calories = round( max(1200, TDEE + adjustment) )      ← never below 1,200 kcal
```

STEP 4 — SPLIT INTO MACROS

Goal	Protein per kg of body weight	Why
Fat loss	2.2 g	Highest — protects muscle in a calorie deficit
Muscle gain	2.0 g	Supports new muscle
Maintenance (and default)	1.6 g	
Endurance	1.4 g	
Flexibility	1.2 g	

```
protein_g = round(weight_kg × protein_per_kg)
fat_g     = round(calories × 25% ÷ 9)                ← fat is 25% of calories, 9 kcal/g
carbs_g   = round((calories − protein_g×4 − fat_g×9) ÷ 4)   ← carbs fill the rest, 4 kcal/g
```

Worked example — male, 178 cm, 75 kg, 25 years, muscle gain, moderately active

Step	Calculation	Result
BMR	10×75 + 6.25×178 − 5×25 + 5 = 750 + 1,112.5 − 125 + 5	1,742.5
TDEE	1,742.5 × 1.45	2,526.6
Calories	2,526.6 + 300, rounded	**2,827 kcal**
Protein	75 × 2.0	**150 g**
Fat	2,827 × 0.25 ÷ 9 = 78.5	**79 g**
Carbs	(2,827 − 600 − 711) ÷ 4	**379 g**

This exact case is one of the app's automated tests, so the formula cannot drift silently.

6.2 Logging a meal

Meals are logged per slot: Breakfast, Lunch, Snack, Dinner. There are four ways in, and all of them end at the same review screen.

1. **Describe it in words →** `parse-meal`
   e.g. "two eggs, toast with butter, black coffee". The model returns a description plus calories, protein, carbs and fat for a regular portion.

2 **Take or pick a photo** → `parse-meal-photo`

The vision model first answers two questions: **is this food?** and **is it appropriate?** If either answer is no, the photo is rejected with a friendly message and no numbers are produced. Otherwise it names what it sees and estimates nutrition for the portion shown.

3 **Reuse a past photo ("Eaten this before?")**

A strip of up to 20 previously photographed meals. Tapping one reuses its numbers and photo without another model call.

4 **Ask for a suggestion** → `suggest-meal`

Covered in 6.3.

PORTION SIZE

The model always estimates a regular portion; small and large are applied afterwards with fixed multipliers. This keeps repeated requests consistent, because asking a model to "make it bigger" gives different answers each time.

Portion	Multiplier
Small	× 0.7
Regular	× 1.0
Large	× 1.4

REVIEW, THEN SAVE

The estimate is shown with **editable** calorie, protein, carb and fat fields. Nothing is written to the database until the user confirms. Only then does `saveMeal` insert into `meal_logs` (if a photo was used, it is uploaded to private storage at this moment, not earlier, so abandoned photos are never stored). The confirmation screen shows the saved values, not the original estimate. **+10 XP.**

WHEN THE MODEL CAN'T BE REACHED

Path	Fallback
Text	A generic estimate for the meal slot, scaled by portion: Breakfast 455 kcal (32P/48C/14F), Lunch 605 (46/58/18), Snack 305 (18/38/9), Dinner 420 (38/32/11). Still goes through review, so the user can correct it.
Photo	No fallback. There is no honest offline guess for what is in a photo, so the user is asked to retry or type the meal instead.

6.3 Meal suggestions

"Not sure what to eat?" takes an optional free-text preference ("something light", "high protein, no dairy", "I only have eggs and rice"). The app sends it along with:

- the user's **dietary restrictions**, marked as rules the model must never break;
- their **remaining budget for today** — calorie and protein targets minus everything already logged today — with an instruction to fit within it.

The model names one specific, real dish and estimates its nutrition. The result goes to the same review screen as any other estimate. If the function is unreachable, a generic slot estimate is offered instead, labelled as generic.

6.4 Water

Daily target **2,000 ml**. Quick-add buttons of 250 ml and 500 ml (presets up to 1,000 ml). Each tap writes one row to `water_logs`. Today's total feeds the recovery score.

6.5 Nutrition summaries

Figure	How it's calculated
Average daily calories (7 days)	Total calories per calendar day over the last 7 days, averaged over *only the days that have logs* — so an unlogged day doesn't drag the average to zero
Days logged (last 7)	Number of distinct days with at least one meal
Days logged this month	Distinct days with at least one meal since the 1st
Calorie ring over 100%	Drawn as a second pass around the same ring, with a shadowed tip showing how far over target the day went (e.g. 3,000 of 2,461 kcal = 122%)

NOT BUILT YET

A full meal-by-meal diet plan (the nutrition equivalent of the workout plan) does not exist. Today the app sets targets, estimates and logs meals, and suggests individual meals.

7 The adaptive system

How CLAWW turns what you did into what you do next. This is the core of the product, so this section goes through every signal, formula and threshold.

7.1 The loop

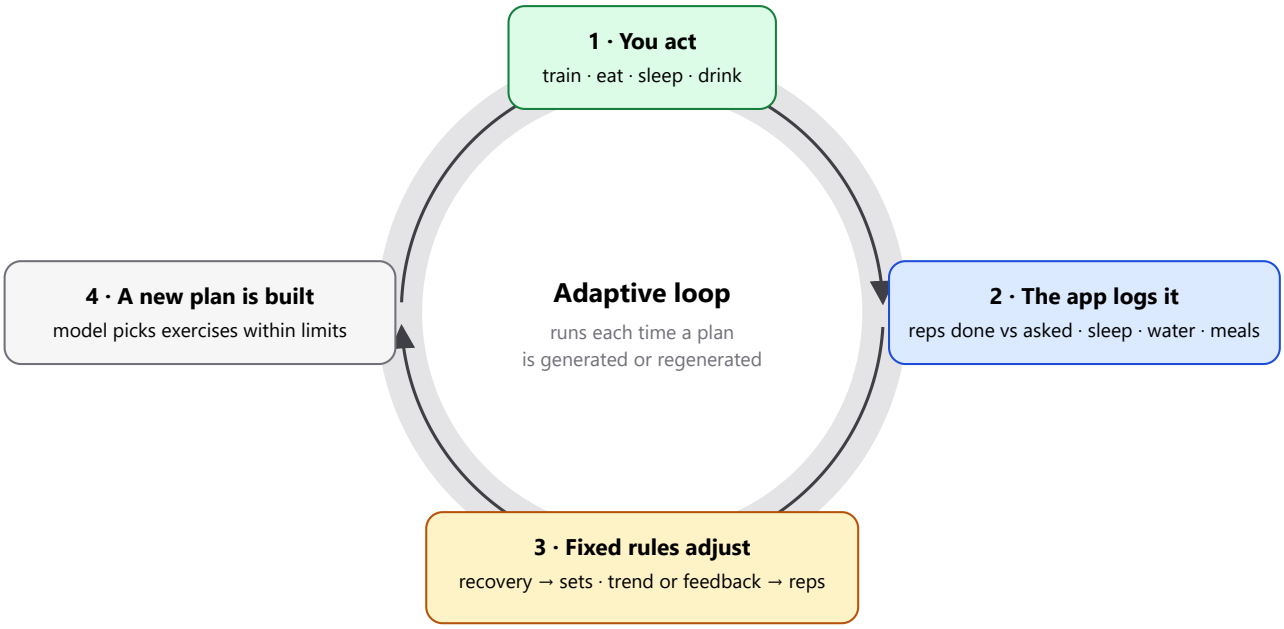


Figure 4 — The model sits only in stage 4, and only chooses exercise names. Every adjustment in stage 3 is deterministic code.

7.2 The three adaptation channels

Channel	Signal	What it changes	Size of change
A · Recovery	Recovery score band (sleep, rest, fatigue, water, meals)	Maximum sets per exercise	Low: −1 · Moderate: 0 · High: +1 (never above the experience cap)
B · Performance	Reps achieved ÷ reps prescribed over the last 10 logged sets	The whole rep window	±2 reps
C · Feedback	Regenerate reason: too hard / too easy / wrong focus	Rep window, or exercise selection	±3 reps, or a different split

They are applied in a fixed order on top of the base range from Section 4:

```
base range (goal → experience → age)
   → Channel A: recovery adjusts max sets
   → Channel C if a reason was given, otherwise Channel B: adjusts reps
   → prompt → model → validate → ground → clamp to this final range
```

7.3 Channel A — Recovery

THE RECOVERY SCORE

A 0–100 estimate of how ready you are to train, computed identically on the server (`_shared/recovery.ts` , used for generation) and on the phone (`lib/data.ts` , used for Home).

```
score = clamp( sleep + rest_gap − fatigue + hydration + nutrition + 20 ,  0, 100 )  → rounded
```

Term	Formula	Range	Source
Sleep	min(hours ÷ 8, 1) × 35	0 – 35	Most recent sleep log (0 if none)
Rest gap	min(hours since last set ÷ 24, 1) × 25 — full 25 if you have never trained	0 – 25	Most recent workout set
Fatigue	intensity × 3, where intensity = clamp(set_number × reps ÷ 10, 0, 10)	0 – 30 (subtracted)	Most recent workout set
Hydration	min(water today ÷ 2,000 ml, 1) × 10	0 – 10	Today's water logs
Nutrition	10 if any meal logged today, else 0	0 or 10	Today's meal logs
Base	constant	20	—

Perfect everything with no fatigue gives 35 + 25 + 10 + 10 + 20 = **100**. Maximum fatigue with nothing else gives 20 − 30 = −10, which is clamped to **0**.

BANDS

Score	Band	Effect on the next plan
0 – 39	Low	Maximum sets reduced by 1 (never below the goal's minimum); the model is told to land lower in each range
40 – 70	Moderate	No change
71 – 100	High	Maximum sets increased by 1 (never above the experience cap); the model is told to land higher

Worked example — recovery score

Slept 6 hours. Last set logged 30 hours ago was set 3 with 10 reps. Drank 1,000 ml today. Logged breakfast.

Term	Calculation	Points
Sleep	6 ÷ 8 × 35	26.25
Rest gap	30 h ≥ 24 h → full	25.00
Fatigue	intensity = 3 × 10 ÷ 10 = 3 → 3 × 3	−9.00
Hydration	1,000 ÷ 2,000 × 10	5.00
Nutrition	meal logged	10.00
Base		20.00
Total → band		**77 → High**



Positive terms add to 86; fatigue takes 9 away, leaving 77. Dashed lines mark the Low/Moderate (40) and Moderate/High (70) boundaries.

HOW THE BAND CHANGES SETS

```
if band = Moderate:  no change
delta   = -1 (Low) or +1 (High)
maxSets = min( experienceCap,  max(minSets, maxSets + delta) )
minSets = min( minSets, maxSets )
```

So an intermediate lifter going for muscle gain (3–4 sets) gets 3–3 when recovery is Low and stays at 3–4 when High, because their cap is 4. An advanced lifter (3–5) gets 3–4 when Low and stays at 3–5 when High.

7.4 Channel B — Performance trend

Used only when no regenerate reason was given. It reads the last 10 logged sets and compares what you did with what you were asked to do.

```
keep only sets where both reps_achieved and reps_prescribed are recorded, and prescribed >
0
if fewer than 3 such sets:  hold  (not enough evidence yet)

ratio = average of (reps_achieved ÷ reps_prescribed) over those sets

ratio  < 0.85   →  DOWN by 2 reps   (you were consistently falling well short)
ratio ≥ 1.05   →  UP   by 2 reps   (you were consistently meeting or beating it)
otherwise      →  hold
```

Direction	Change to the rep window	Guard rails
Down	min −2, max −2	Minimum reps never below 1; max never below min
Up	min +2, max +2	Final clamp still applies

Worked example — trend

Last sets (prescribed → achieved)	Ratio
10 → 12, 10 → 11, 10 → 11, 12 → 13, 12 → 13	1.20, 1.10, 1.10, 1.08, 1.08
Average	**1.11 → UP**

A 6–12 rep window becomes 8–14 on the next generation. If instead the average had been 0.80 (e.g. asked for 10, doing 8), it would become 4–10.

7.5 Channel C — Explicit feedback

When the user regenerates, they pick a reason. Explicit feedback always wins over the inferred trend, because it is direct information from the person.

Reason	Effect
Too hard	Rep window moves down by **3** (larger than the automatic ±2, because the user said so directly)
Too easy	Rep window moves up by **3**
Wrong focus	Numbers are left to the trend (Channel B). The model is told the previous split and focus were wrong and to choose a meaningfully different split and exercise selection

A regeneration with any reason also bypasses the 30-second cooldown, so the user never gets the plan they are complaining about handed straight back.

7.6 Adaptive scheduling

Separately from the plan contents, the Workouts tab adapts which day it suggests: whichever day follows the last one you completed *or skipped*. Skipping a day doesn't leave you stuck on it, and the calendar shows completed and skipped days.

7.7 Everything the adaptive system reads

Input	Where it comes from	Feeds
Reps achieved vs prescribed	Set confirm card, every set	Channel B, fatigue term
Weight achieved vs prescribed	Set confirm card	Sent to the model as history; volume; weight carry-over between sets
Time of last set	`workout_logs.completed_at`	Rest-gap term
Sleep hours	Morning check-in (bedtime + wake time)	Sleep term
Water today	Water widget	Hydration term
Any meal today	Meal logging	Nutrition term
Regenerate reason	Regenerate card	Channel C
Completed / skipped days	Session finish	Next-day suggestion

WHY IT'S BUILT THIS WAY

Every adjustment is small, bounded, and explainable in one sentence. A language model left to "adapt" a plan on its own would be inconsistent between runs, could not be tested, and could drift into unsafe volumes. Keeping the model out of the numbers means each rule can be unit-tested — the recovery formula has 11 hand-calculated tests, including the exact band boundaries (39, 40, 70, 71).

7.8 What the adaptive system does not do (yet)

HONEST LIMITS

- **It adapts at the next generation, not continuously.** A new sleep log does not rewrite a plan you've already started; the change shows up when you next generate or regenerate. This is deliberate — silently changing a workout you've committed to is worse than waiting.
- **It doesn't progress weights automatically.** Weights are logged and sent to the model as history, and each set starts from your last weight, but there is no rule that adds load week to week.
- **Recovery is an estimate, not a physiological measurement.** It uses sleep, rest and logging habits — not heart-rate variability or resting heart rate like a Whoop or Oura. The app's wording says "estimate" for this reason.
- **Steps don't count towards recovery.** Steps live only on the phone (Health Connect or Core Motion) and are never stored on the server, so the server-side formula can't use them without inventing a number.
- **Fatigue only looks at the single most recent set**, and uses its set number × reps as a load proxy, because there is no effort (RPE) field yet.

8 Progress, XP and streaks

Every point comes from a real logged action, recorded in an append-only ledger.

XP

Action	XP	Ledger reason
Complete a set (or log an extra session)	5	`set_completed`
Log a meal	10	`meal_logged`
Log sleep	10	`sleep_logged`
Generate a plan	50	`plan_generated`

Each award inserts one row into `xp_events`; the total is the sum, via the `user_xp` view. Awarding XP never blocks the action — if the XP insert fails, the meal or set is still saved. The number on screen updates instantly (optimistically) and matches what the server records.

Levels and tiers

```
level          = floor(XP ÷ 100) + 1
progress bar   = XP mod 100, out of 100
tier           = changes every 5 levels
CLAWW Score    = min(XP, 1000)
```

Levels	Tier name
1–5	Getting Started
6–10	Building Momentum
11–15	Committed
16–20	Dedicated
21+	Elite

Activity streak

The number of consecutive calendar days with at least one sleep log, meal log or workout set. If nothing has been logged yet today, counting starts from yesterday — so a streak doesn't show as broken before the day is over.

Achievements

Badge	Earned when
🎯 First Steps	Any sleep, meal or plan exists
🔥 Week Warrior	Streak of 7 days or more
💪 Planner	At least one plan generated
🥗 Nourished	10 or more meals logged
🌙 Night Owl	Any logged bedtime between 23:00 and 04:00
⚡ Consistent	5 or more sleep logs
🚀 CLAWW Legend	500 XP or more

The Tracker tab

State	Shows
Locked (fewer than 3 workout days)	CLAWW Score, workouts completed out of 3, and what unlocks next. Streak, volume and goals wait for enough data
Unlocked (3+ workout days)	CLAWW Score, streak, workouts this month, volume, goals, a volume insight, a nutrition section (7-day average calories, days logged), and a shareable text report

A "workout day" is any calendar day with at least one logged set. There is no session ID, so one day of training counts as one workout, the same rule the streak uses.

9 Home, sensors and reminders

The Home rings

Ring	Value
Outer (blue)	Recovery score, 0–100
Middle (white)	Sleep ring (currently drawn from the same recovery score)
Inner (green)	Calories eaten ÷ calorie target — **not capped**, so an over-target day shows the overflow tip
Centre	Daily goal % = average of the three rings (nutrition capped at 100% for this average)

Rings are drawn with Skia, which allows a true angular gradient: dark at the start, bright at the tip. Past 100%, the ring continues around the same circle rather than adding a second ring, with a small shadow ahead of the tip — the same visual language as Apple's Activity rings.

Steps and calories burned

Platform	Source	Notes
Android	Health Connect (steps + active calories, read-only)	Android's own step sensor can't answer "steps since midnight", so Health Connect is required. If it isn't installed, the app offers a link to install it
iPhone	Core Motion via the phone's pedometer	Supports date-range queries directly

When Health Connect provides real active calories, those are shown. Otherwise the app estimates from steps:

```
calories ≈ steps × body_weight_kg × 0.0005        (70 kg if weight isn't set)
e.g. 8,000 steps × 70 kg × 0.0005 = 280 kcal
```

Step data stays on the phone; it is never sent to the server or the language model.

Sleep check-in

The user enters bedtime and wake-up time with scroll wheels; hours are calculated from the two. One log per day is shown as "Logged". +10 XP.

Reminders

Local notifications scheduled on the phone (not server push). Permission is only requested after the first completed workout, via a soft prompt, never on first launch. They can be switched on or off in Settings.

Reminder	Time	Tapping it opens
Meal	13:00	Meal logging
Workout	18:00	Workouts tab
Sleep	22:00	Home

10 Reliability, safety and cost control

Model fallback chains

Used for	Order tried
Text (plans, meal text, suggestions)	1. `openai/gpt-oss-120b` → 2. `openai/gpt-oss-20b`
Vision (meal photos)	`qwen/qwen3.8-27b` only — the single image model enabled on the account

A model is skipped and the next one tried only for failures a different model might not have: permission blocked (401/403), model not found (404), rate limited (429), or a server error (5xx). A malformed request (other 4xx) stops immediately, since every model would reject it the same way. All model calls ask for JSON-only output with temperature 0.4.

Failure logging

Every failure — validation failed, grounding emptied the plan, permission blocked, rate limited, and so on — is written to `generation_failures` with the function name and details. The plan response itself also says whether it was really generated, so "is generation working?" can be answered directly. Unexpected crashes are reported to Sentry once a Sentry project is connected.

Usage caps

Kind	Per user per day	Covers
plan	20	generate-plan
meal	50	parse-meal, parse-meal-photo, suggest-meal

A single database function checks and increments the counter in one statement, so concurrent requests can't overshoot. Settings shows today's usage.

Privacy and account control

Feature	How it works
Row-level security	Every table restricts rows to their owner. The app uses the user's own login token for all ordinary reads and writes
Meal photos	Private bucket; each user can only access files in their own folder; displayed via links that expire after 1 hour
Download my data	Exports every table the user owns as one JSON file
Delete account	Two independent confirmations (type DELETE in the app; the server checks again). Deleting the user cascades to every table automatically
What the model sees	Profile preferences, injury text, workout history, meal text and photos, remaining macros. Never step data, never email address

11 Honest limits and what's next

Current limits

Area	Limit
Injury handling	Keyword filter on exercise names, not a structured contraindication model
Progression	Rep ranges adapt; weight loads don't progress automatically
Recovery	Behavioural estimate; no HRV/heart-rate input; steps not included
Adaptation timing	Takes effect at the next generation, not mid-plan
Nutrition	Targets and per-meal estimates only; no full diet plan
Exercise guidance	No how-to descriptions, target muscles or alternatives shown yet
Offline	Set logs are not queued if the connection drops mid-workout
Vision	Only one image model available, so photo estimation has no backup model

On the roadmap

Phase	Item
Ship 10.1	Structured movement-pattern contraindications replacing the injury keyword filter
Ship 10.2	Photo size ceiling and per-minute rate limit
Ship 10.3	Offline queue for set logs
Ship 10.5	Accessibility pass
Ship 11.1	Read sleep from Health Connect; write workouts back
Ship 11.2	Per-exercise form guidance, written by people rather than generated
Ship 11.3	Progress and insights screen from data already collected
Ship 11.5	Subscriptions, using the usage caps as the natural free/paid line
Ship 12	Device QA, 14-day closed test with 12 testers, store listing, staged launch

A Appendix

File map

File	What's in it
`supabase/functions/generate-plan/index.ts`	The whole plan pipeline: rep tables, frequency table, age rules, injury filter, adaptive adjustments, prompt, validation, fallback plan, clamping
`supabase/functions/_shared/recovery.ts`	Recovery score formula and bands (server)
`supabase/functions/_shared/planning.ts`	Equipment tiers and catalog grounding
`supabase/functions/_shared/groq.ts`	Model chains, error types, retry rules
`supabase/functions/_shared/usageCap.ts`	Daily caps
`supabase/functions/parse-meal/`, `parse-meal-photo/`, `suggest-meal/`	Meal estimation and suggestion
`app/lib/nutrition.ts`	Calorie and macro formula
`app/lib/data.ts`	Every database query: logging, volume, calorie burn, streak, XP, achievements, recovery (client copy), export
`app/lib/workoutSession.tsx`	Live workout state machine
`app/lib/steps.ts`	Health Connect / Core Motion steps and calories
`app/components/ui/ProgressRing.tsx`	Skia ring renderer, including overflow
`database/schema.sql`, `seed_exercises.sql`	All tables, security policies, cap function; the 183-exercise catalog

Glossary

BMR	Basal metabolic rate — calories burned at complete rest
TDEE	Total daily energy expenditure — BMR scaled by activity
MET	Metabolic equivalent — an activity's energy cost relative to sitting still (1.0)
Rep range / window	The allowed minimum and maximum repetitions per set
Split	How training days divide the body: full body, upper/lower, push/pull/legs
Grounding	Removing anything the model returned that isn't in the list it was given
Clamping	Forcing a number into an allowed range after the fact
Fallback	A safe, pre-written result used when generation fails
Row-level security	A database rule that limits each user to their own rows
Edge Function	Server code running on Supabase, used for anything needing a model or extra permissions
Zod schema	A strict description of the expected JSON shape, used to validate model output

Ratio (Channel B)	Reps achieved ÷ reps prescribed, averaged over recent sets